

Cost-Aware Repository Change Localization with Sparse Impact Planning and Bounded Verification

MSc Research Proposal (V1.5, supervisor-ready / print-candidate)

Ahmed Ehab

2026-09-17

Abstract

Large language models (LLMs) that edit a software repository must first decide which files a requested change affects. This decision is costly in two ways: an explicit file-by-file impact policy over a repository-scale candidate universe serializes hundreds of decisions per call, and a second-stage reasoner that revisits the scope can multiply inference cost by orders of magnitude.

Existing localization methods typically emit one affected-file set and stop; they neither expose the files the first pass chose to leave unchanged as a first-class object nor reason about how much a second look should cost. The precise gap addressed by this thesis is the absence of an explicit, budget-controlled treatment of the first-pass residual: *which omitted files deserve a second look, and at what cost.*

Central identity of the thesis:

This thesis studies whether a cheap sparse file-impact policy can reduce repository-wide localization effort while a bounded second look recovers important files the first pass chose not to change.

We therefore propose and evaluate a two-stage design: a sparse, preserve-by-omission first-pass impact policy, followed by a bounded omission-recovery stage that ranks the first-pass omitted candidates with cheap structural and lexical evidence and inspects only the top- B under a hard per-task budget B . The experimental setting uses real commits from two Python/Django repositories (djangoCMS, Saleor), parent-only inputs, an analytic-Random control, task-level bootstrap uncertainty, and the observed historical change set used strictly as an evaluation proxy.

The strongest result is confirmatory: on the authorized djangoCMS internal test (80 tasks), the frozen composite ranker recovers Sparse-omitted files in the observed historical change-set proxy above the analytic-Random baseline at every budget point $B \in \{1, 3, 5, 10\}$ ($B=5$: 0.165 vs 0.028, delta +0.137, interval excluding zero), so the fixed-budget recovery signal is scientifically confirmed on that test. A second-repository DEVELOPMENT replication on Saleor shows the same direction ($B=5$: 0.237 vs 0.006) and is reported as development evidence only. The signal is predominantly lexical (BM25); the thesis makes no graph-novelty claim.

Status and scope boundaries are kept explicit: the fixed-budget result is confirmed; adaptive budget allocation is a future development program (P2), not a completed contribution; and generalization so far covers Python/Django repositories, with cross-language evaluation deferred to future external-validity work. One scope limitation applies throughout: localization is evaluated against the observed historical diff as a proxy for impact, never against semantic ground truth.

1 Background, motivation, and problem

Repository-level coding agents and impact planners must select affected files before editing. Two problems motivate the thesis. First, *impact inference*: did the model identify the right files? Second, *impact-policy representation*: can a complete file-action policy be expressed without spending output budget on hundreds of repeated *preserve* decisions? Our controlled study shows that an explicit full policy at a 16K completion cap serializes on average 144 decision records per candidate universe, while a sparse policy serializes on average 4.9 records (a $\sim 96.6\%$ reduction), with large associated reductions in completion tokens and cost.

Second-stage reasoning is expensive: a graph-guided agent evaluated under a shared protocol consumed on the order of a million tokens per task on the same held-out tasks where our sparse first pass stays below a few thousand tokens. If a cheap first pass already localizes the bulk of affected files, the question is whether a *bounded* second look can recover missed files without paying the full always-on cost.

Given a repository revision P (parent commit), a change request q , and a candidate universe $U(P)$ visible at P , a localization system produces an affected-file set $\hat{S}(P, q)$; the hidden observed change set $S^*(P)$ is used for evaluation only. The observed historical diff is an *observed change-set proxy*, never semantic impact gold (Section 6).

2 Central thesis hypothesis and gap

Hypothesis. A sparse/cheap first-pass localization policy followed by a bounded omission-recovery mechanism can dominate both cheap-localization-only and always-on expensive reasoning on the correctness–cost Pareto frontier, for the repositories and protocols we evaluate.

Everything else supports this story:

- *Preserve-by-Omission* = representation / first-pass contribution;
- *BM25 / classical static change-impact analysis* = cheap baselines;
- *graph-guided agent* = expensive system-level context;
- *task-level omission-risk scorer* = investigated and currently rejected on development evidence;
- *candidate-level bounded verification* = primary next mechanism;
- *second and third repositories* = generalization.

Task-level RiskScorer success is *not* a thesis requirement. Existing work covers several individual components; this thesis evaluates *their combination* as a sparse first pass plus bounded omission recovery under matched budgets (Section 4).

3 Research questions and expected contributions

- RQ1.** How does sparse explicit impact-policy representation affect inference/output cost and file-selection fidelity?
- RQ2.** How accurately can cheap lexical/static methods localize impacted files relative to LLM planners under shared protocols?
- RQ3.** Can bounded second-stage verification recover missed files in the observed historical change-set proxy more efficiently than always-on or random matched-budget verification?
- RQ4.** How do these correctness–cost trade-offs transfer across repositories and ecosystems?

If task-level risk modeling remains unsupported, RQ3 is answered by the candidate-level mechanism. *Expected contributions:* (1) sparse explicit impact-plan representation with controlled evidence; (2) a reproducible real-commit impact-localization dataset and protocol; (3) shared-protocol, budget-matched cheap/classical/LLM/agent comparison under common file-level metrics and explicit budgets; (4) a bounded/selective-compute verification protocol (candidate-level by default); (5) cross-repository correctness–cost evidence; (6) reproducibility artifacts and leakage-control methodology.

No positive algorithmic result is promised.

4 Positioning against related work

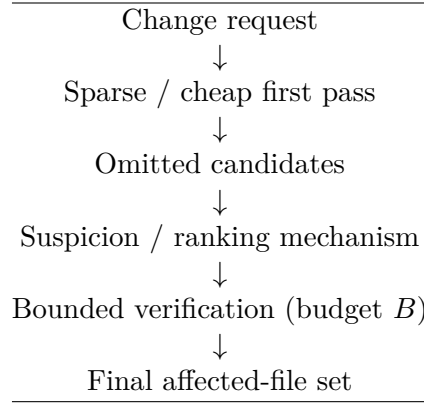
Existing work covers several individual components. This thesis evaluates their combination. Verified related lines (primary sources, 2026-09-16/17):

- *Classical change-impact / ripple effect (RIPPLE line)*. RIPPLE = a known seed $\rightarrow$ recall expansion $\rightarrow$ precision refinement [1]. This thesis differs: an explicit repository-wide sparse policy $\rightarrow$ a rejected residual set $\rightarrow$ bounded reconsideration of rejected candidates.
- *Change-Patterns Mapping* (TSE 2022) [2]: boosts/reranks the initial impact set from historical patterns. This thesis allocates *new inspection to the excluded residual set* rather than reranking the selected set.
- *ArtifactSync*: a demo-level mechanistic analogue (uncertainty- driven deeper context) but a different post-commit synchronization task, not file-impact localization.
- *Modern transform + program-dependence-graph impact analysis* [3]: graph+LLM impact analysis is active; our graph is one optional verifier.
- *Selective-compute / efficiency+verification priors* [4, 5]: selective retrieval / efficient verification at repository level for code-completion/generation tasks, guarding generic “selective escalation is novel” claims.
- *Repository-level coding and retrieval agents* [6, 7, 8, 9, 10]: LLM agents and retrieval-conditioned generation over whole repositories; this thesis evaluates a cheap sparse first pass against shared-cost context.
- *Graph-guided issue localization* [11, 12, 13]: expensive graph-guided competitors (LocAgent, GraphLocator, LLM-driven iterative code-graph search).
- *Repository-level code graphs for LLMs* [14]: RepoGraph (ICLR 2025, primary source verified) builds an AST-derived repository code graph that guides LLM localization/generation; it is a navigation aid for agents, not a sparse policy with a bounded verifier.
- *Classical / history-learned change-impact predictors* [15, 16, 17, 18]: dependency- and history-based impact prediction and industrial studies.

Graph, history, and selective computation are *not* novelty claims by themselves. The comparison matrix (Section 7, two compact tables) records whether each line has an explicit file policy, sparse omission representation, cheap first pass, omission recovery, hard/matched budget, real-commit evaluation, and cross-repository evidence. Where novelty is only defensible once experiments support it, it is marked *candidate novelty — not yet claimed*.

5 Proposed two-stage architecture

1. **Sparse/cheap first-pass impact policy:** explicit file-level policy with preserve-by-omission serialization.
2. **Candidate-level bounded verification** (primary): rank the first-pass omitted candidates with independent structural / retrieval / history evidence, and verify only those under a hard per-task budget B . Compare with always-verify and random matched-budget verification.



6 Dataset construction, labels, and leakage controls

- **Real history, parent-only inputs.** Inference uses the parent commit P ; the target T is hidden. Live HEAD is never used.
- **Eligibility and exclusions.** Single-parent commits with a meaningful intent, touching only production source files; tests, configurations, docs, migrations, and generated files are excluded; intent-path leakage is a hard exclusion.
- **Deduplication.** Exact-proxy-set / shared-reference / suspected- related changes are removed with frozen deterministic rules.
- **Observed change-set proxy.** $P \rightarrow T$ changed production files are an *observed* proxy, never semantic impact gold. Localization claims are scoped to historical-file recovery unless semantic adjudication supports more.
- **Splits.** Metadata-only, deterministic split frozen before any model result. For djangoCMS V2-LARGE: development, an untouched internal test, and a reserve. The v1 40 are LEGACY-EXPOSED and never reused as untouched confirmation.

7 Experimental design, baselines, budgets, metrics, statistics

7.1 Budget curve, not a single point

The primary scientific object is the *budget curve* $B \in \{0, 1, 3, 5, 10\}$; $B=0$ is Sparse alone. $B=5$ is a convenient reference operating point, not a scientifically privileged single claim. We also report the normalized sensitivity $B/|\text{omitted set}|$ as a secondary view.

7.2 Analytic Random control

For ranking-only experiments the Random control is the analytic hypergeometric expectation: for a task with N_t omitted candidates, M_t Sparse-observed FNs, and budget B , $\mathbb{E}[X_t] = B \cdot M_t / N_t$

(with B clipped to N_t). The comparison is per-task observed recovery vs this analytic expectation, aggregated with task-level bootstrap intervals. Random repetitions are never independent tasks.

7.3 Primary endpoint

False-Negative Recovery Rate / Omission Recovery Rate @ B : per task, recovered Sparse-observed FNs within B / all Sparse-observed FNs. Secondary: absolute recall gain, FNR reduction, fraction of Oracle@ B gap closed, candidates inspected per recovered FN, precision/F1 (safety metrics), runtime, index/build cost.

7.4 Anchors (per B)

Sparse/ $B=0$; Analytic Random; BM25; Path-token; Graph/static; BM25+Graph-Neighbor Composite (historical label: Classical-CIA; the frozen primary; short form: *BM25+GraphNeighbor*); history/co-change (where parent-visible history is available); one fixed simple hybrid (0.5 BM25 + 0.5 graph neighbor, a rank-equivalent redundant control — confirmed by the ranker-identity audit, not an independent baseline); Oracle (ranking headroom); InspectAll (exhaustive). *Oracle@ $B \neq$ InspectAll*: Oracle is ranking headroom; InspectAll is exhaustive reconsideration.

Stage boundaries. Ranking, verification, and final localization are distinct stages and are never conflated: the ranker orders first-pass omitted candidates for inspection; the verifier accepts or rejects individual candidates under the budget B ; and the final affected-file set is assembled only after bounded verification. The proposal reports each stage separately with its own metric.

7.5 Experimental Design at a Glance

Study	Purpose	Repo / Split	Method	Unit	Budget / Reps	Primary metrics	Status
Controlled	sparse-vs-full cost	djangoCMS / controlled	Full vs Sparse	task	cap 16K / 3 reps	tokens, records, cost, P/R/F1	Done
Historical	selection fidelity	djangoCMS / held-out	Full vs Sparse	task	cap 16K / 3 reps	P/R/F1/FNR	Done
Agent	expensive context	djangoCMS / held-out	graph-guided agent	task	per-call	P/R/F1, cost	Done
Cheap baselines	zero-LLM signal	djangoCMS / dev	BM25 / graph	task	K curve	P/R/F1	Done
Task risk	risk predictability	djangoCMS / dev	features	task	3 reps	AUROC/AUPRC	Negative / closed
Candidate recovery	omission recovery	djangoCMS / dev	composite ranker + verifier	cand.	$B \in \{0..10\}$	FN-recovery	DEV replicated; CON-FIRMED (2026-09-17)
Cross-repo transfer	transfer	Saleor / dev	frozen method	task	B curve	per-repo	Saleor DEV replication complete

Table 1: Experimental Design at a Glance (internal IDs appear in traceability notes only). Status as of 2026-09-17.

7.6 Related-work comparison matrix

The matrix promised in Section 4. Rows are the related lines verified from primary sources; the last row is this proposal. To keep the tables readable at A4, the columns are split into two compact tables (design/structure, then evidence/position). “Yes/No” entries state what each line provides or evaluates; “–” means not applicable or not stated in the primary source. The two tables must be read together; they do not overlap.

7.6.1 Design and structure

Line	Input	Output unit	First pass	Second stage	Hist.	Graph	Budget
Classical / history CIA	change request or seed change	affected file / module set	yes	no	some	yes	no
Agentless	issue + repo	localized files + patch	yes	no	no	no	no
CodePlan	change request + repo	file-level edit plan	yes	partial	no	partial	no
RepoCoder / repo retrieval	query + retrieved context	code completion	retrieve	iter. gen.	no	no	no
LocAgent	issue + repo	localized file set	agent search	agent refine	no	yes	no
GraphLocator	issue + repo	localized file set	graph expand	causal iter.	no	yes	no
RepoGraph (ICLR 2025)	repo	graph navigation	graph build	plug-in to agents	no	yes	no
This proposal	request + parent + universe	affected file set	sparse policy	bounded top- B	opt.	bin.	yes

Table 2: Related-work comparison matrix (1/2): design and structure. Verified primary sources; entries state what each line provides.

7.6.2 Evidence and position

7.7 Statistics and multiple-comparison discipline

Task is the independent unit; candidate rows are nested. One primary endpoint (recovery@ B); one primary arm-vs-Random comparison; secondary endpoints labeled; task-level paired bootstrap; no post-hoc feature-family winner; exploratory analyses reported as exploratory.

8 Preliminary evidence

All numbers are from audited studies in this repository; they are development or controlled evidence with their bounds.

- **Representation effect (controlled).** At a 16K completion cap, Sparse serialized on average **4.9** decision records vs **144.0** for Full (**~96.6%** reduction), with large completion-token and cost reductions. No universal semantic-superiority claim.

Line	False-negative recovery	Real commits	Cross-repo	Main difference
Classical / history CIA	partial (rerank)	some	no	no sparse policy; no budgeted reconsideration of the rejected residual
Agentless	no	SWE-bench	yes	one-shot localize-then-repair; no explicit budget on file selection
CodePlan	no	internal + benchmarks	some	explicit file plan but no omission representation; no bounded residual verification
RepoCoder / repo retrieval	no	benchmarks	some	output is code, not an affected-file decision; no omission recovery
LocAgent	no	SWE-bench style	yes	expensive graph-guided agent; no hard per-task budget; no omitted-set recovery
GraphLocator	no	benchmarks	yes	causal graph expansion; expensive; no budget curve
RepoGraph (ICLR 2025)	no	SWE-bench; Cross-CodeEval	yes	graph is a navigation aid for agents; no sparse policy, no verifier
This proposal	yes (primary object)	djangoCMS + Saleor	yes (2 repos; NestJS future)	sparse explicit file policy + bounded, budget-matched reconsideration of the rejected residual

Table 3: Related-work comparison matrix (2/2): evidence and position. “This proposal” row states the intended/planned design; see the experiment tables for which parts are done versus future.

- **Historical real-commit study.** 60 cells over 10 held-out real djangoCMS commits: selection is difficult, FNR is high, and there is no Full-vs-Sparse superiority claim (paired deltas’ intervals cross zero). Sparse is substantially cheaper.
- **Graph-guided agent comparison.** System-level shared-protocol context: 402 calls / 32.8M tokens / ~\$9.93 estimated on 10 tasks, with 5/10 non-usable outcomes (2 timeout + 1 context-length + 2 completed-but-empty). Not a reproduction of the published fine-tuned result.
- **Cheap/static development baselines.** BM25 provides a meaningful zero-LLM signal; graph-ranked seeds are comparable; a genuinely classical dependency-propagation CIA baseline (a separate script, not the frozen composite) is implemented on 150 development cases.
- **Task-level omission-risk investigation.** Live Sparse development inference (90 cells; then 431 calls with a fail-closed stop at the 2.5M-token ceiling). A task-level RiskScorer is not justified: the v1 signal does not replicate within the new development cohort and the pooled signal is a candidate-universe-size artifact.
- **Candidate-level bounded verification (Route B, development).** A frozen BM25+Graph-Neighbor composite ranker (historical label: Classical-CIA; exact formula = normalized BM25 + binary graph-neighbor, per the ranker-identity audit) recovers Sparse-omitted missed files in the observed historical change-set proxy above the analytic-Random baseline across the whole budget curve $B \in \{1, 3, 5, 10\}$, with task-level bootstrap intervals excluding zero at every point (e.g. at $B=5$: pooled macro recovery 0.163 vs random 0.029, interval $[+0.09, +0.18]$), 5/5 development folds positive, and no omitted/universe-size artifact. A small verifier pilot (30 calls, ~\$0.002) found the frozen ranker places the

recoverable missed files within the top- B (Oracle-in-top- $B \approx 1.0$) and the verifier recovers ~ 0.86 – 1.0 of them, so the dominant loss is first-pass omission.

- **Saleor DEVELOPMENT transfer replication (2026-09-17).** A clean Saleor DEVELOPMENT Sparse run (150 tasks $\times$ 3 reps = 450 cells; 446 valid / 4 failed; 7,316,986 tokens / $\sim$ \\$2.31; 0 truncations; no Saleor-specific tuning) enabled a frozen Route-B transfer test on 149 evaluable tasks: the composite beats analytic Random at every B ($B=5$: 0.237 vs 0.006, delta +0.231, CI [+0.180, +0.287]; 5/5 folds positive; no size artifact). This is a **DEVELOPMENT transfer replication, NOT a confirmatory result**; Saleor INTERNAL_TEST/RESERVE remain sealed.
- **djangoCMS confirmatory result (2026-09-17, CONFIRMS).** The frozen confirmatory protocol ran on the authorized djangoCMS V2 INTERNAL_TEST (80 tasks) under the frozen V2 packet and budget (560 calls / 1,470,174 tokens / $\sim$ \\$0.51; 0 failures; 0 excluded): the composite beats analytic Random at every B ($B=1$ 0.059 vs 0.006; $B=3$ 0.110 vs 0.017; $B=5$ 0.165 vs 0.028, delta +0.137, CI [+0.075, +0.205]; $B=10$ 0.267 vs 0.055), with task-level bootstrap intervals excluding zero at every B and no size artifact. **Fixed-budget Route B is scientifically CONFIRMED on djangoCMS INTERNAL_TEST.** The confirmatory test is permanently used and will not be reused for adaptive-budget (P2) selection.
- **Incremental-evidence ablation (zero API, 2026-09-17).** Composite-minus-BM25 paired task-level deltas are small with bootstrap CIs including zero at most B on both repositories (djangoCMS $B=5$ +0.003 [−0.015, +0.019]; Saleor $B=5$ +0.003 [−0.017, +0.024]); the replicated cross-repo signal is **predominantly lexical (BM25)**. The thesis does not claim graph novelty.
- **Classical/history evidence.** A parent-visible history/co-change arm also beats analytic Random where history is available; the BM25+Graph-Neighbor composite remains the frozen primary.

These results motivate a larger development dataset, protection of an untouched internal test, and the bounded-verification design; they are not rewritten as a success.

9 Cross-repository generalization

Stage 1 djangoCMS V2-LARGE (within-repo large evidence + untouched internal test) $\rightarrow$ Stage 2 Saleor (different real system, same Python/Django stack) $\rightarrow$ Stage 3 NestJS (cross-language / cross-framework evidence). Each stage states its claim boundary. NestJS is preferred but not forced; a predeclared eligible-pool yield criterion and a backup protocol are frozen.

9.1 Saleor DEVELOPMENT transfer replication (completed, 2026-09-17)

The Saleor DEVELOPMENT sparse run (150 $\times$ 3 = 450 cells; 446 valid / 4 failed; 7,316,986 tokens / $\sim$ \\$2.31; 0 truncations; no Saleor-specific tuning) and the frozen Route-B transfer test (149 evaluable tasks) replicate the bounded omission-recovery signal on a second, much larger repository ($B=5$ composite 0.237 vs analytic Random 0.006; delta +0.231, CI [+0.180, +0.287]; 5/5 folds positive; no size artifact). This is **DEVELOPMENT transfer replication only — NOT confirmatory**. Saleor INTERNAL_TEST/RESERVE are sealed and were never opened. Per-repository primary; no djangoCMS+Saleor pooling as the headline.

9.2 djangoCMS confirmatory result (2026-09-17, CONFIRMS)

The frozen confirmatory protocol ran on the authorized djangoCMS V2 INTERNAL_TEST (80 tasks) under the frozen V2 packet and budget (560 calls / 1,470,174 tokens / ~\$0.51; 0 failures; 0 excluded): the frozen BM25+Graph-Neighbor Composite recovers Sparse-omitted files in the observed historical change-set proxy above the analytic Random control across the whole budget curve ($B=1$ 0.059 vs 0.006; $B=3$ 0.110 vs 0.017; $B=5$ 0.165 vs 0.028; $B=10$ 0.267 vs 0.055), with task-level bootstrap intervals excluding zero at every B and no size artifact. **The fixed-budget Route-B result is scientifically CONFIRMED on djangoCMS INTERNAL_TEST.** The confirmatory test is permanently used and will not be reused for adaptive-budget (P2) selection; P2 is evaluated on development data only, with Saleor INTERNAL_TEST sealed as a possible future P2 confirmation set.

9.3 Scope contract

CORE MSc: Preserve-by-Omission representation evidence; fixed-budget Route-B development + the confirmed djangoCMS INTERNAL_TEST result; semantic-proxy audit; djangoCMS; Saleor as secondary replication if completed. *Conditional / Future*: adaptive B_t (P2, development-data exploration, Nov 2026 – Mar 2027; **NOT complete — a development research program, not a proven contribution**); learned budget allocation; NestJS cross-language (**future external-validity work only**, planned April 2027, conditional on the suitability gate and TypeScript-extractor readiness); end-to-end regeneration correctness; persistent memory; richer causal-graph verifier.

10 Threats, contingency, and timeline

10.1 Threats to validity

Internal: the observed proxy is not semantic gold; leakage control is mandatory; no hidden-test peeking. External: n and single-repository generalization; V2 and cross-repository evidence are the mitigation, with per-repository primary metrics. Statistical: low negatives, multiple comparisons, wide random bands — addressed by pre-registration, class-balance gates, analytic-Random controls, and bootstrap uncertainty. Representation: LLM serialization cost spans providers; both recorded and frozen-pricing cost are reported.

10.2 Semantic-proxy validity

Because the historical diff is only a proxy, we prepare a stratified development-only sample with evidence packets for later human adjudication.

The protocol is docs/SEMANTIC_PROXY_AUDIT_PROTOCOL.md.

Machine-assisted notes are never semantic gold.

10.3 Contingency

If candidate-level bounded verification does not beat analytic Random at the frozen budget, the negative result is frozen and reported; no complex verifier is invented.

10.4 Future work: adaptive / cost-sensitive verification allocation

P2 (adaptive budget) is **NOT complete**. It is a development research program for November 2026 – March 2027 that runs *after* the fixed Route-B confirmation (which is done and CONFIRMED) and is evaluated on development data only; it is a program, not a proven contribution. The program covers Shichao-Zhang-inspired algorithms (Learning- k , one-step KNN, cost-sensitive

KNN, adaptive-neighborhood) [19, 20, 21, 22] plus competitors and analogues from selective prediction / abstention, learning-to-defer, cascaded inference, optimal stopping, budgeted retrieval, value-of-information, and interpretable adaptive stopping. We sketch the objective: $B_t^* = \arg \min_B [C_{\text{verify}}(B) + \rho \mathbb{E}[\text{remaining omission loss} \mid \text{evidence}, B]]$, with ρ a sensitivity, not a claimed monetary truth. This is NOT yet a contribution; if no pre-registered policy justifies itself on development data, P2 closes negative and the fixed-B thesis (already confirmed) stands. Status: CONDITIONAL; see the P2 pre-registration note.

10.5 Timeline (2026-11 to 2027-10)

Window	Planned work
2026-11	Fixed Route-B confirmation (djangoCMS INTERNAL_TEST; CONFIRMED 2026-09-17) + P2 literature/formulation freeze
2026-12	Simple Shichao-inspired adaptive-budget policies (score-gap, marginal-score, cost-ratio stopping) on development data
2027-01	Cost-sensitive / one-step / demand-driven adaptive-budget analogues (development data only)
2027-02	Newly discovered competitor/alternative algorithm reproduction (selective/cascade/optimal-stopping families)
2027-03	P2 comparison/freeze/pre-registration on development; negative closure if not justified; Saleor INTERNAL_TEST only if pre-registered
2027-04	Optional external-validity extension (NestJS or predeclared backup <i>only if</i> the suitability gate passes AND the TypeScript extractor is ready); future external-validity work, no forced cross-language claim
2027-05	Consolidated statistical analysis; Pareto/frontier analysis; semantic-audit integration; final threat-to-validity revision
2027-06	Thesis chapters/results writing; artifact cleanup; reproducibility package; supervisor review round 1
2027-07	Thesis full draft; corrections; final experimental closure; submission-ready research manuscript draft
2027-08	Target research/thesis completion; supervisor review round 2; final figures/tables/appendices; defense/seminar preparation
2027-09	Publication revision/submission window; reviewer-style internal audit; contingency for delayed replication or manuscript resubmission
2027-10	Final buffer: journal/conference resubmission, thesis formatting/administrative closure, archival artifact release

Table 4: Work schedule, November 2026 – October 2027. July–August 2027 is the intended substantive research/thesis completion; September–October 2027 is an honest publication/revision/administrative buffer, not fabricated work. November 2026 – March 2027 is the P2 adaptive-budget exploration program (development data only).

Intended substantive research/thesis completion: **2027-07/08**. The 2027-09/10 window is a publication, revision, and administrative buffer. No experiments are fabricated merely to fill months. P2 (adaptive budget) is a conditional development research program and is **NOT complete**: if it does not justify a candidate on development data, it is closed as negative and the fixed Route-B result (already confirmed on djangoCMS INTERNAL_TEST) remains the thesis basis. NestJS / NextJS / cross-language extension is **future external-validity work only**, conditional on its frozen gates and on TypeScript-extractor readiness.

11 References

All entries below were verified against primary sources (arXiv API / Crossref) on 2026-09-16/17. Preprints are explicitly marked. No fabricated or placeholder citations remain; the source bibliography is `references.bib` (V1.5; the RepoGraph entry was verified for V1.5 from arXiv:2410.14684).

References

- [1] F. M. Haney. Module connection analysis — a tool for scheduling software debugging activities. In *AFIPS Joint Computer Conferences*, 1972. Classical change-impact / ripple-effect line (concept-level).
- [2] Yuan Huang, Jinyu Jiang, Xiapu Luo, Xiangping Chen, Zibin Zheng, Nan Jia, and Gang Hua. Change-patterns mapping: A boosting way for change impact analysis. *IEEE Transactions on Software Engineering*, 48(7):2476–2495, 2022.
- [3] Yanfu Yan, Nathan Cooper, Kevin Moran, Gabriele Bavota, Denys Poshyvanyk, and Stephen Rich. Enhancing code understanding for impact analysis by combining transformers and program dependence graphs. <https://arxiv.org/abs/2607.23355>, 2026. Preprint (arXiv:2607.23355). Transformers + program-dependence-graph impact analysis.
- [4] Di Wu, Wasi Uddin Ahmad, Dejiao Zhang, Murali Krishna Ramanathan, and Xiaofei Ma. Repoformer: Selective retrieval for repository-level code completion. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.
- [5] Qianhui Zhao, Li Zhang, Fang Liu, Xiaoli Lian, Qiaoyuanhe Meng, et al. FastCoder: Accelerating repository-level code generation via efficient retrieval and verification. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025.
- [6] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. <https://arxiv.org/abs/2407.01489>, 2024. Preprint (arXiv:2407.01489). Localize-then-repair without an agent loop.
- [7] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, D. C. Vageesh, Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, B. Ashok, and Shashank Shet. Codeplan: Repository-level coding using LLMs and planning. In *Proceedings of the 46th International Conference on Software Engineering (ICSE)*, 2024. arXiv:2309.12499; ICSE 2024.
- [8] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. arXiv:2303.12570; EMNLP 2023. Repository memory / retrieval-conditioned generation line.
- [9] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024. arXiv:2404.05427; ISSTA 2024. AST-level search + repair loop.
- [10] Qinyu Luo, Yining Ye, Shihao Liang, Zhong Zhang, Yujia Qin, Yaxi Lu, Yesai Wu, Xin Cong, Yankai Lin, Yingli Zhang, Xiaoyin Che, Zhiyuan Liu, and Maosong Sun. RepoAgent: An LLM-powered open-source framework for repository-level code documentation generation and agentic quality management. <https://arxiv.org/abs/2402.16667>, 2024. Preprint (arXiv:2402.16667). Repository memory / agent line.

- [11] Zhaoling Chen, Xiangru Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. LocAgent: Graph-guided LLM agents for code localization. <https://arxiv.org/abs/2503.09089>, 2025. Preprint (arXiv:2503.09089). A graph-guided localization agent; evaluated in this project under a shared protocol (P5) at an upstream commit (pin 4935b557).
- [12] Wei Liu, Chao Peng, Pengfei Gao, Aofan Liu, Wei Zhang, Haiyan Zhao, and Zhi Jin. GraphLocator: Graph-guided causal reasoning for issue localization. <https://arxiv.org/abs/2512.22469>, 2025. Preprint (arXiv:2512.22469). LLM causal-issue-graph localization.
- [13] Zhonghao Jiang, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. Issue localization via LLM-driven iterative code graph searching. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025.
- [14] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. RepoGraph: Enhancing AI software engineering with repository-level code graph. <https://arxiv.org/abs/2410.14684>, 2024. arXiv:2410.14684; ICLR 2025 (arXiv comment). Repository-level code-graph navigation plug-in for LLM localization/generation.
- [15] Hani Abdeen, Khaled Bali, Houari Sahraoui, and Bruno Dufour. Learning dependency-based change impact predictors using independent change histories. *Information and Software Technology*, 67:220–233, 2015.
- [16] Haruya Iwasaki, Tsuyoshi Nakajima, Ryota Tsukamoto, Kazuko Takahashi, and Shuichi Tokumoto. A software impact analysis tool based on change history learning and its evaluation. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2022.
- [17] Markus Borg, Krzysztof Wnuk, Bjorn Regnell, and Per Runeson. Supporting change impact analysis using a recommendation system: An industrial case study in a safety-critical context. *IEEE Transactions on Software Engineering*, 43(9):812–829, 2017.
- [18] Romina Etezadi, Sallam Abualhaija, Chetan Arora, and Lionel Briand. LLM-driven cost-effective requirements change impact analysis. <https://arxiv.org/abs/2511.00262>, 2025. Preprint (arXiv:2511.00262). Cost-aware requirements change-impact analysis (requirements unit).
- [19] Shichao Zhang. Challenges in KNN classification. *IEEE Transactions on Knowledge and Data Engineering*, 34(10):4663–4675, 2022.
- [20] Shichao Zhang and Jianping Li. KNN classification with one-step computation. *IEEE Transactions on Knowledge and Data Engineering*, 2023.
- [21] Shichao Zhang. Cost-sensitive KNN classification. *Neurocomputing*, 391:234–242, 2020.
- [22] Shichao Zhang, Jianping Li, and Yuling Li. Reachable distance function for KNN classification. *IEEE Transactions on Knowledge and Data Engineering*, 2023.